

SKILL WEEK-11

1 . To connect different program modules, validate their interfaces, test end-to-end execution, handle errors, improve reliability, and refactor components.

Aim : To develop a modular C program where separate components communicate through well-defined interfaces, verify the complete execution flow, handle errors properly, and improve code reliability through refactoring.

Program :

```
GNU nano 7.2
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);

#endif
```

```
GNU nano 7.2
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}
```

```
GNU nano 7.2
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 10;
    int b = 5;

    printf("Module Integration Test\n");
    printf("-----\n");

    printf("A = %d\n", a);
    printf("B = %d\n\n", b);

    printf("Addition      : %d\n", add(a, b));
    printf("Subtraction    : %d\n", subtract(a, b));
    printf("Multiplication : %d\n", multiply(a, b));

    printf("\nEnd-to-end execution completed successfully.\n");

    return 0;
}
```

Output :

```

jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano calculator.h
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano calculator.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano main_module.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -c calculator.c -o calculator.o
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -c main_module.c -o main_module.o
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc calculator.o main_module.o -o module_test
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./module_test

```

Module Integration Test

A = 10

B = 5

Addition : 15

Subtraction : 5

Multiplication : 50

End-to-end execution completed successfully.

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc main_module.c -o test_error

```

/usr/bin/ld: /tmp/ccH4ZHiLy.o: in function 'main':

main_module.c:(.text+0x75): undefined reference to 'add'

/usr/bin/ld: main_module.c:(.text+0x9a): undefined reference to 'subtract'

/usr/bin/ld: main_module.c:(.text+0xbf): undefined reference to 'multiply'

collect2: error: ld returned 1 exit status

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ls -l *.c *.h *.o

```

```

-rw-r--r-- 1 jhanasri jhanasri 169 Sep 22 13:41 calculator.c
-rw-r--r-- 1 jhanasri jhanasri 139 Sep 22 13:39 calculator.h
-rw-r--r-- 1 jhanasri jhanasri 1468 Sep 22 13:42 calculator.o
-rw-r--r-- 1 jhanasri jhanasri 1018 Sep 18 03:50 client.c
-rw-r--r-- 1 jhanasri jhanasri 1376 Sep 18 03:32 ls_grp.c
-rw-r--r-- 1 jhanasri jhanasri 420 Aug 13 11:45 main_loop.c
-rw-r--r-- 1 jhanasri jhanasri 465 Sep 22 13:42 main_module.c
-rw-r--r-- 1 jhanasri jhanasri 2408 Sep 22 13:42 main_module.o
-rw-r--r-- 1 jhanasri jhanasri 1677 Sep 18 03:20 producer_consumer.c
-rw-r--r-- 1 jhanasri jhanasri 1275 Sep 18 03:48 server.c
-rw-r--r-- 1 jhanasri jhanasri 726 Sep 18 03:58 signal_handler.c
-rw-r--r-- 1 jhanasri jhanasri 1851 Aug 17 13:42 task1_wait.c
-rw-r--r-- 1 jhanasri jhanasri 2388 Aug 13 11:56 task2_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 1793 Sep 18 04:13 task2_single_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 3056 Sep 18 04:38 task3_double_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 940 Aug 17 06:02 task3_fork.c
-rw-r--r-- 1 jhanasri jhanasri 1661 Aug 7 08:09 task3_input.c
-rw-r--r-- 1 jhanasri jhanasri 2297 Aug 17 04:59 task4_escape.c
-rw-r--r-- 1 jhanasri jhanasri 1976 Aug 7 08:19 task4_history.c
-rw-r--r-- 1 jhanasri jhanasri 951 Aug 17 13:32 task4_state.c

```

Observation

The separate C modules were compiled and linked successfully using well-defined interfaces. The calculator operations produced the expected results, and the missing-module test generated undefined-reference errors.

Result

The modular program successfully demonstrated module integration, interface validation, error detection, and end-to-end execution.

2 . To detect runtime errors, generate meaningful error messages, log failures, handle invalid syntax, recover gracefully, and test different error scenarios.

Aim : To implement a C program that detects and handles runtime and syntax errors, displays appropriate error messages, records failures in a log file, and continues execution safely after recoverable errors.

Program :

```
GNU nano 7.3
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

void log_error(const char *message)
{
    FILE *log = fopen("error.log", "a");

    if (log == NULL)
    {
        perror("Cannot open error log");
        return;
    }

    fprintf(log, "ERROR: %s\n", message);
    fclose(log);
}

int main()
{
    char input[100];
    int number;
    FILE *file;

    printf("Runtime Error and Syntax Handling\n");
    printf("-----\n");

    /* Test invalid file access */
    file = fopen("missing_file.txt", "r");
```

```

GNU nano 7.2
/* Test invalid file access */
file = fopen("missing_file.txt", "r");

if (file == NULL)
{
    printf("Error: Unable to open missing_file.txt\n");
    printf("Reason: %s\n", strerror(errno));

    log_error("Failed to open missing_file.txt");

    printf("Recovery: Continuing program execution...\n\n");
}
else
{
    fclose(file);
}

/* Test invalid numeric input */
printf("Enter an integer: ");

if (fgetc(input, sizeof(input), stdin) == NULL)
{
    printf("Error: Input could not be read.\n");
    log_error("Input reading failed");
    return 1;
}

char extra;

if (sscanf(input, "%d %c", &number, &extra) != 2)
{
    printf("Syntax Error: Invalid integer input.\n");
    printf("Recovery: Please enter a valid integer.\n");

    log_error("Invalid integer syntax");

    number = 0;
}

printf("Processed value: %d\n", number);

printf("\nProgram recovered successfully.\n");
printf("Testing completed.\n");

return 0;

```

Output :

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_error_handling
Enter an ineger: abc
Syntax Error: Invalid integer input.
Recovery: Please enter a valid integer.
Processed value: 0

Program recovered successfully.
Testing completed.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cat error.log
ERROR: Failed to open missing_file.txt
ERROR: Failed to open missing_file.txt
ERROR: Invalid integer syntax
jhanasri@LAPTOP-1RN1M8E3:~/OSSPP$ ls -l missing_file.txt
-rw-r--r-- 1 jhanasri jhanasri 108 Sep 22 13:51 error.log
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cat error.log
ERROR: Failed to open missing_file.txt
ERROR: Failed to open missing_file.txt
ERROR: Invalid integer syntax
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ rm -f error.log
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_error_handling
Runtime Error and Syntax Handling

Error: Unable to open missing_file.txt
Reason: No such file or directory
Recovery: Continuing program execution...
Enter an integer: 25
Processed value: 25
Program recovered successfully.
Testing completed.
jhanasri@LAPTOP-1RN1M8E3:~/OSSPP$ |

```

Observation

The program detected invalid integer input and missing-file errors and generated appropriate error messages. The failures were recorded in `error.log`, while valid input was processed successfully.

Result

The program successfully demonstrated runtime and syntax error handling, logging, and graceful recovery from recoverable errors.

3 . To run Valgrind, identify memory leaks and memory errors, fix the errors, release allocated resources, verify cleanup, and document the findings.

Aim : To analyze dynamic memory usage in a C program using Valgrind, detect memory leaks and invalid memory operations, correct the errors, and verify that all allocated memory is properly released.

Program :

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *name;
    char *message;

    printf("Valgrind Memory Test\n");
    printf("-----\n");

    name = malloc(50 * sizeof(char));
    message = malloc(100 * sizeof(char));

    if (name == NULL || message == NULL)
    {
        printf("Memory allocation failed.\n");
        free(name);
        free(message);
        return 1;
    }

    strcpy(name, "OSSP Student");
    strcpy(message, "Memory allocation successful.");

    printf("Name      : %s\n", name);
    printf("Message   : %s\n", message);

    /* Release allocated memory */
    free(name);
    free(message);
}

```

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *data;

    data = malloc(5 * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        data[i] = i + 1;
    }

    printf("Data: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");

    /* Intentionally missing free(data) */
    return 0;
}

```

```

GNU nano 7.2
VALGRIND MEMORY ANALYSIS REPORT

Task:
Run Valgrind, identify leaks, fix memory errors,
release resources, verify cleanup, and document findings.

Program:
task_valgrind.c

Tool:
Valgrind

Initial Test:
Dynamic memory was allocated using malloc().

Problem Identified:
Memory that is allocated dynamically must be released
using free() before program termination.

Fix Applied:
free() was used to release allocated memory.
Pointers were then set to NULL.

Final Verification:
Valgrind reported:
All heap blocks were freed -- no leaks are possible.

Error Summary:
0 errors from 0 contexts.

Result:
Memory was successfully allocated, used, released,
and verified using Valgrind.

```

Output :

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_valgrind.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -g task_valgrind.c -o task_valgrind
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_valgrind
Valgrind Memory Test
-----
Name      : OSSP Student
Message   : Memory allocation successful.

Memory released successfully.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind
==506== Memcheck, a memory error detector
==506== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==506== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==506== Command: ./task_valgrind
==506==
Valgrind Memory Test
-----
Name      : OSSP Student
Message   : Memory allocation successful.

Memory released successfully.
==506==
==506== HEAP SUMMARY:
==506==    in use at exit: 0 bytes in 0 blocks
==506==    total heap usage: 3 allocs, 3 frees, 1,174 bytes allocated
==506==
==506== All heap blocks were freed -- no leaks are possible
==506==
==506== For lists of detected and suppressed errors, rerun with: -s
==506== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_valgrind.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -g task_valgrind.c -o task_valgrind
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind
==510== Memcheck, a memory error detector
==510== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==510== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==510== Command: ./task_valgrind
Data: 1 2 3 4 5
==510==
==510== HEAP SUMMARY:
==510==   in use at exit: 20 bytes in 1 blocks
==510==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==510==
==510== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==510==   at 0x484063B: malloc (vg_replace_malloc.c:381)
==510==   by 0x10910C: main (task_valgrind_leak.c:8)
==510==
==510== LEAK SUMMARY:
==510==   definitely lost: 20 bytes in 1 blocks
==510==   indirectly lost: 0 bytes in 0 blocks
==510==   possibly lost: 0 bytes in 0 blocks
==510==   still reachable: 0 bytes in 0 blocks
==510==   suppressed: 0 bytes in 0 blocks
==510==
==510== For lists of detected and suppressed errors, rerun with: -s
==510== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_valgrind_leak.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -g task_valgrind_leak.c -o task_valgrind_leak
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind_leak
==520== Memcheck, a memory error detector
==520== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==520== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==520== Command: ./task_valgrind_leak
==520==
==520== 1 2 3 4 5
==520==
==520== HEAP SUMMARY:
==520==   in use at exit: 8 bytes in 0 blocks
==520==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==520== All heap blocks were freed -- no leaks are possible
==520== For lists of detected and suppressed errors, rerun with: -s
==520== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ valgrind --leak-check=full --show-leak-kinds=all ./task_valgrind
==533== Memcheck, a memory error detector
==533== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==533== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==533== Command: ./task_valgrind
==533==
Valgrind Memory Test
-----
Name      : OSSP Student
Message   : Memory allocation successful.

Memory released successfully.
==533==
==533== HEAP SUMMARY:
==533==      in use at exit: 0 bytes in 0 blocks
==533==    total heap usage: 3 allocs, 3 frees, 1,174 bytes allocated
==533==
==533== All heap blocks were freed -- no leaks are possible
==533==
==533== For lists of detected and suppressed errors, rerun with: -s
==533== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano valgrind_report.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cat valgrind_report.txt
VALGRIND MEMORY ANALYSIS REPORT

Task:
Run Valgrind, identify leaks, fix memory errors,
release resources, verify cleanup, and document findings.

Program:
task_valgrind.c

Tool:
Valgrind

Initial Test:
Dynamic memory was allocated using malloc().

Problem Identified:
Memory that is allocated dynamically must be released
using free() before program termination.

Fix Applied:
free() was used to release allocated memory.
Pointers were then set to NULL.

Final Verification:
Valgrind reported:
All heap blocks were freed -- no leaks are possible.

Error Summary:
0 errors from 0 contexts.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |

```

Observation

Valgrind detected the dynamically allocated memory that was not released. After applying `free()`, the program was tested again, and Valgrind confirmed that all allocated memory was released.

Result

The program successfully identified and fixed memory leaks, with the final verification showing zero memory errors and no leaks.

4 . To design test cases, create test scripts, automate their execution, verify outputs, compare expected and actual results, and generate test reports.

Aim : To implement an automated testing system for a C program that executes multiple test cases, checks the program output against expected results, and generates a test report.

Program :

GNU nano 7.2

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    int a = 10;
    int b = 5;

    printf("ADD: %d\n", add(a, b));
    printf("SUBTRACT: %d\n", subtract(a, b));
    printf("MULTIPLY: %d\n", multiply(a, b));

    return 0;
}
```

```

GNU nano 7.2
#!/bin/bash

echo "Automated Test Execution"
echo "===== "

PASS=0
FAIL=0

echo ""
echo "Test 1: Addition"

EXPECTED="ADD: 15"
ACTUAL=$(./calculator_test | grep "ADD")

if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "PASS"
    PASS=$((PASS + 1))
else
    echo "FAIL"
    echo "Expected: $EXPECTED"
    echo "Actual   : $ACTUAL"
    FAIL=$((FAIL + 1))
fi

echo ""
echo "Test 2: Subtraction"

EXPECTED="SUBTRACT: 5"
ACTUAL=$(./calculator_test | grep "SUBTRACT")

if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "PASS"
    PASS=$((PASS + 1))
else
    echo "FAIL"
    echo "Expected: $EXPECTED"
    echo "Actual   : $ACTUAL"
    FAIL=$((FAIL + 1))
fi

echo ""
echo "Test 3: Multiplication"

```

```

EXPECTED="MULTIPLY: 50"
ACTUAL=$(./calculator_test | grep "MULTIPLY")

if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "PASS"
    PASS=$((PASS + 1))
else
    echo "FAIL"
    echo "Expected: $EXPECTED"
    echo "Actual   : $ACTUAL"
    FAIL=$((FAIL + 1))
fi

echo ""
echo "===== "
echo "Test Summary"
echo "===== "
echo "Passed: $PASS"
echo "Failed: $FAIL"

if [ $FAIL -eq 0 ]; then
    echo "Overall Result: PASS"
else
    echo "Overall Result: FAIL"
fi

```

```

GNU nano 7.2
Test 1: Addition
PASS

Test 2: Subtraction
PASS

Test 3: Multiplication
PASS|

```

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano calculator_test.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra calculator_test.c -o calculator_test
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./calculator_test
ADD: 15
SUBTRACT: 5
MULTIPLY: 50

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano test_script.sh
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ chmod +x test_script.sh
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./test_script.sh
Automated Test Execution
=====

Test 1: Addition
PASS

Test 2: Subtraction
PASS

Test 3: Multiplication
PASS

=====
Test Summary
=====
Passed: 3
Failed: 0
Overall Result: PASS
```

Observation

The automated testing script executed addition, subtraction, and multiplication test cases. All three tests passed, and the generated report was compared with the expected output.

Result

The automated testing system successfully verified all test cases and generated a final report showing **3 passed and 0 failed**.